

Software Engineering

Dr. Md. Nasim Adnan

Internal and External Views of Testing

- Any engineered product (and most other things) can be tested in one of two ways:
- (1) Knowing the specified functions that a product has been designed to perform, tests can be conducted that demonstrate each function is fully operational while at the same time searching for errors in each function.
- (2) Knowing the internal workings of a product, tests can be conducted to ensure that “all gears mesh,” that is, internal operations are performed according to specifications and all internal components have been adequately exercised.
- The first test approach takes an external view and is called black-box testing.
- The second requires an internal view and is termed white-box testing.

Views of Testing

```
graph TD; A[Views of Testing] --> B[White Box Testing]; A --> C[Black Box Testing];
```

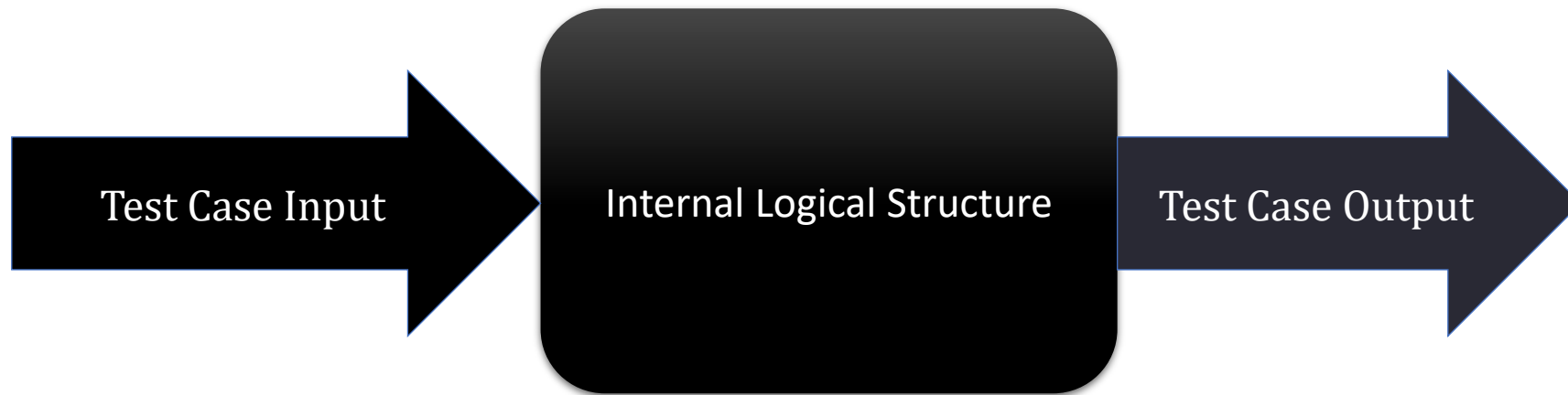
The diagram is a simple tree structure. At the top is a light blue rounded rectangle with a darker blue shadow, containing the text 'Views of Testing'. A vertical line descends from the bottom center of this box and splits into two horizontal lines. Each horizontal line connects to the top center of a light gray rounded rectangle with a darker gray shadow. The left gray box contains the text 'White Box Testing' and the right gray box contains the text 'Black Box Testing'.

White Box
Testing

Black Box
Testing

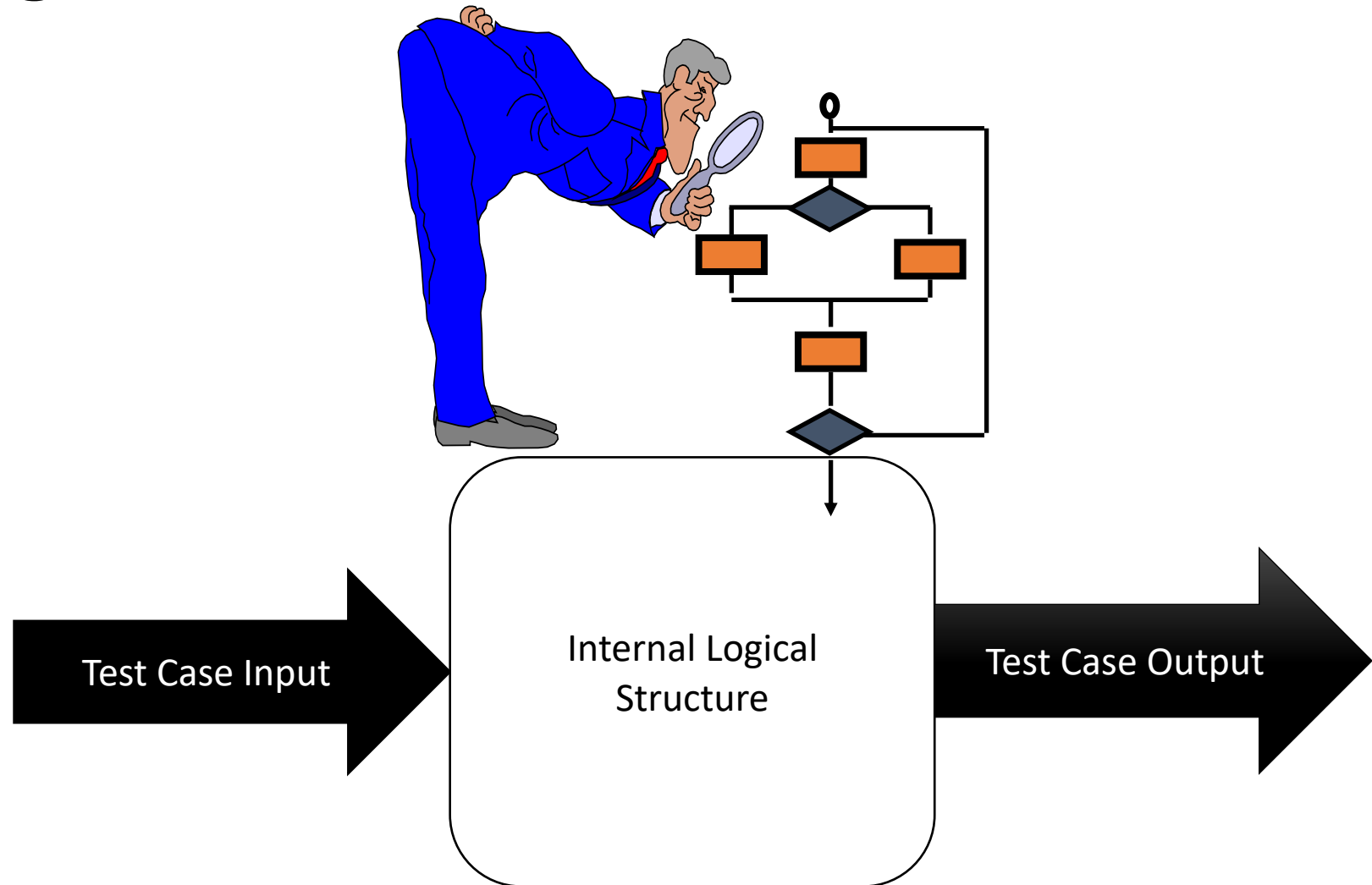
Black-box testing

- Black-box testing alludes to tests that are conducted at the software interface. A black-box test examines some fundamental aspect of a system with little regard for the internal logical structure of the software.



White-box testing

- White-box testing of software is predicated on close examination of procedural detail.
- Logical paths through the software and collaborations between components are tested by exercising specific sets of conditions and/or loops.



- At first glance it would seem that very thorough white-box testing would lead to “100 percent correct programs.” All we need do is define all logical paths, develop test cases to exercise them, and evaluate results, that is, generate test cases to exercise program logic exhaustively.
- Unfortunately, exhaustive testing presents certain logistical problems. For even small programs, the number of possible logical paths can be very large. White-box testing should not, however, be dismissed as impractical. A limited number of important logical paths can be selected and exercised. Important data structures can be probed for validity.

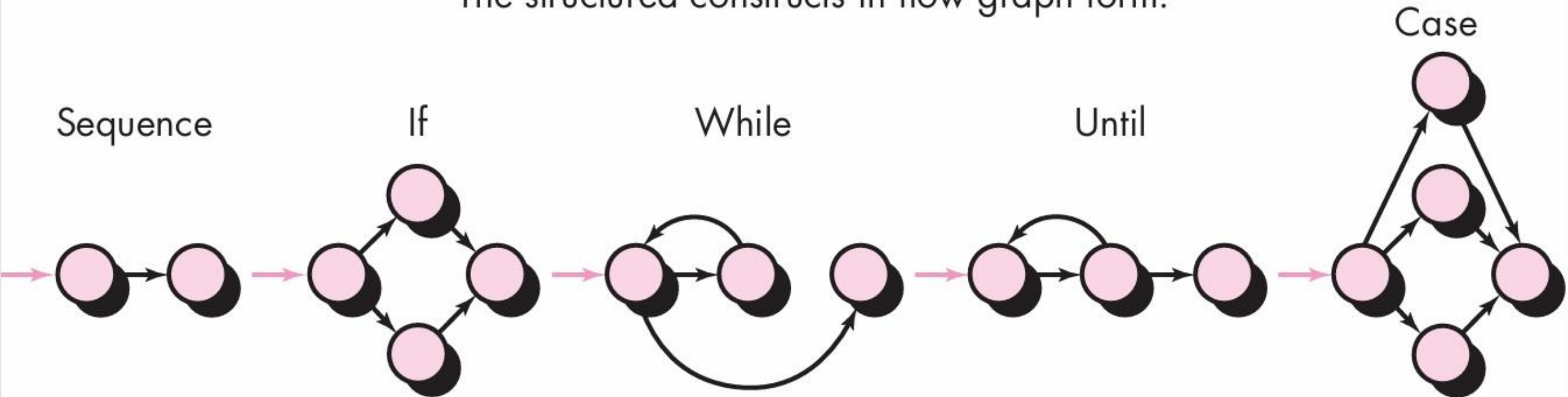
White-Box Testing

- White-box testing, sometimes called glass-box testing, clear box testing, transparent box testing, and structural testing, is a test-case design philosophy that uses the control structure to derive test cases.
- Using white-box testing methods, you can derive test cases that
- (1) guarantee that all independent paths within a module have been exercised at least once
- (2) exercise all logical decisions on their true and false sides
- (3) execute all loops at their boundaries and within their operational bounds
- (4) exercise internal data structures to ensure their validity

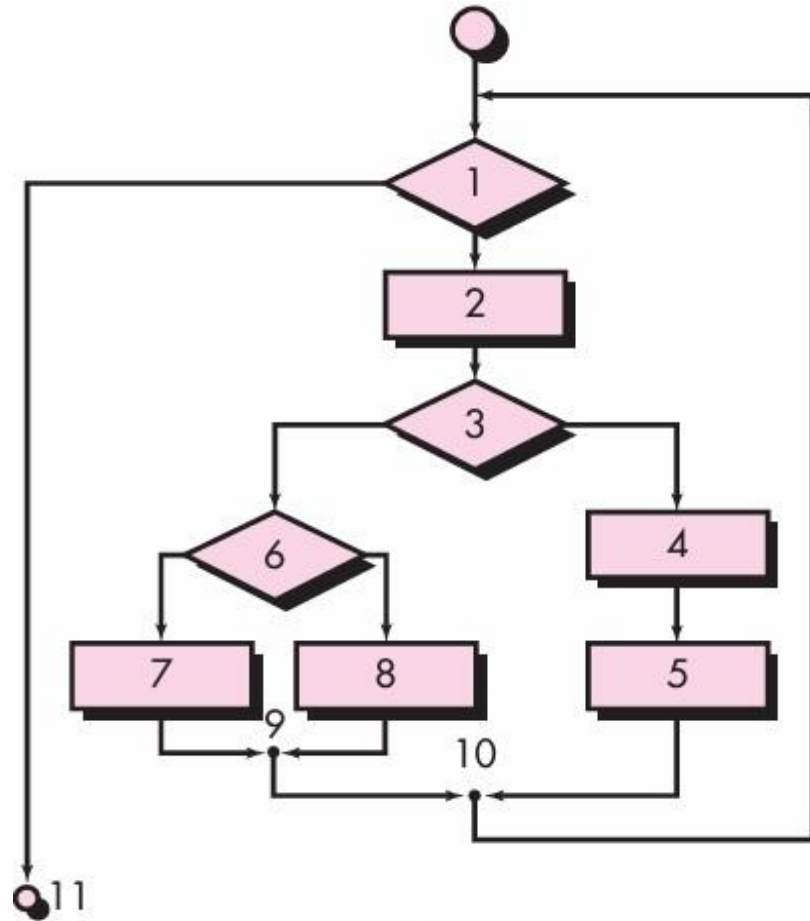
Basis Path Testing

- Before we consider the basis path testing, a simple notation for the representation of control flow, called a flow graph (or program graph) must be introduced.
- The flow graph depicts logical control flow using the notation illustrated in the next figure.

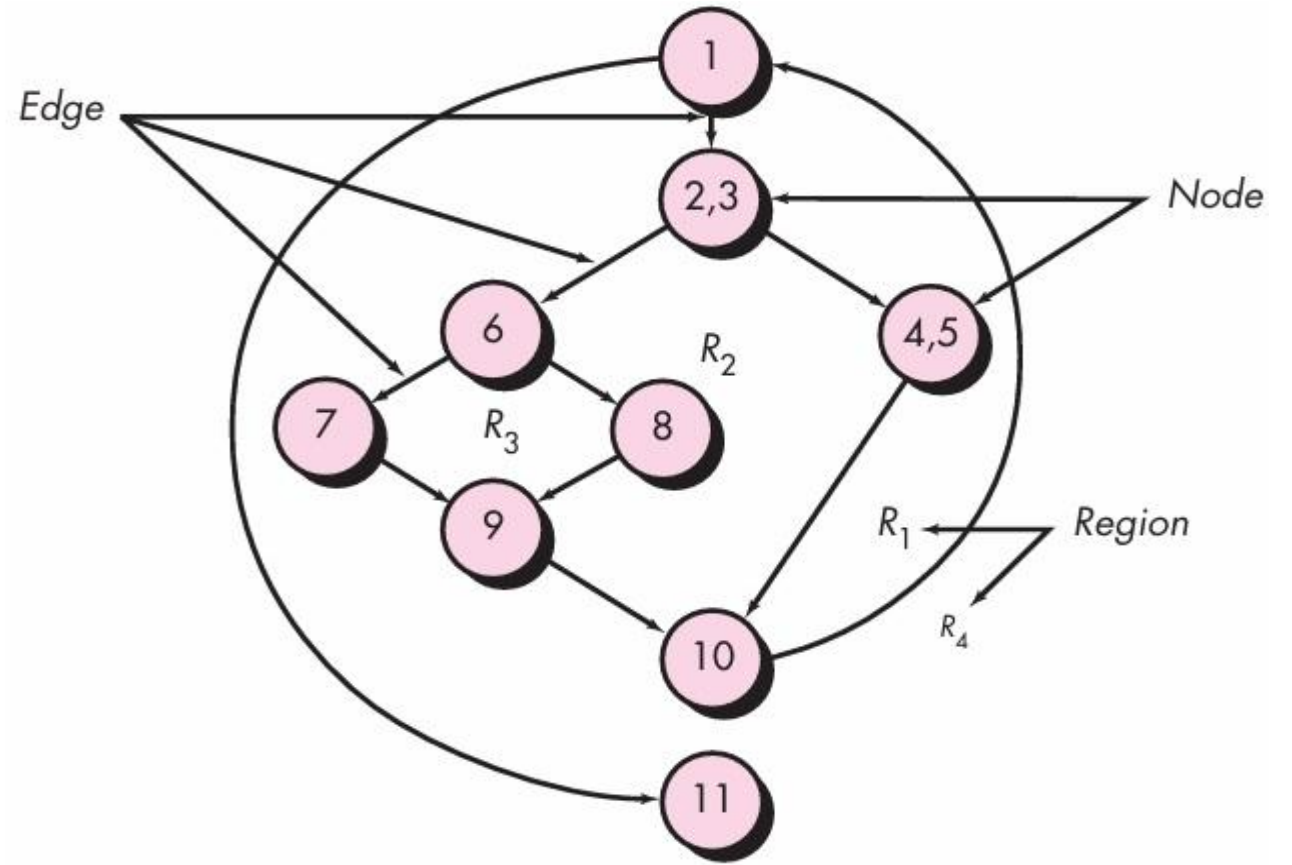
The structured constructs in flow graph form:



Flow Graph Notation



(a)



(b)

Independent Program Paths

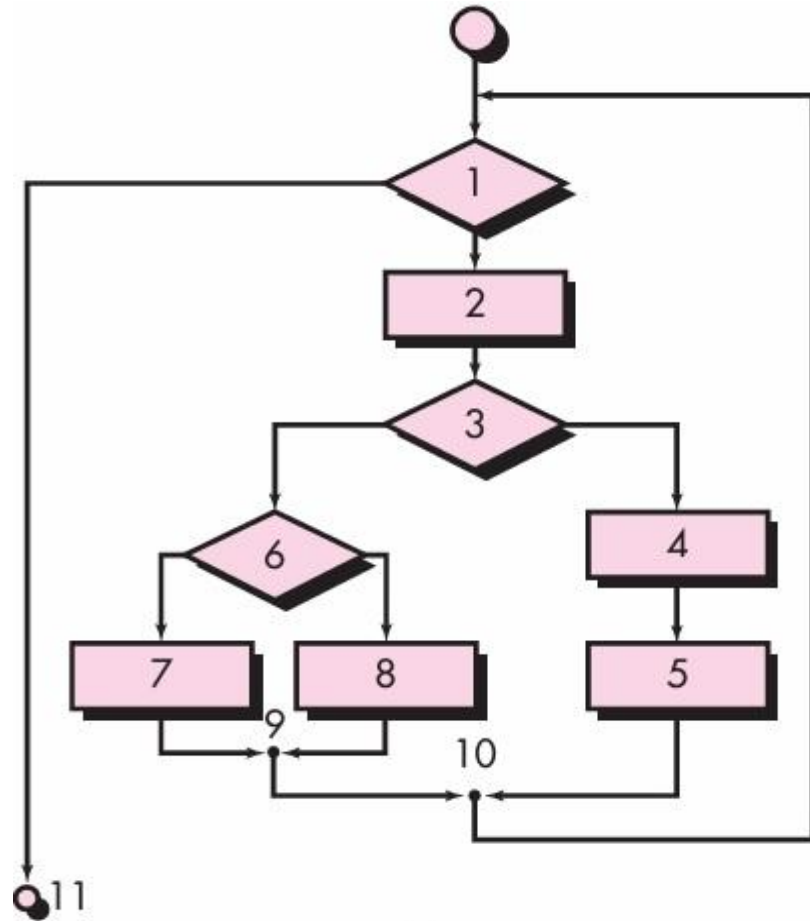
- An independent path is any path through the program that introduces at least one new set of processing statement.
- When stated in terms of a flow graph, an independent path must move along at least one edge that has not been traversed before the path is defined.
- For example, a set of independent paths for the flow graph illustrated in the previous figure is:
 - Path 1: 1-11
 - Path 2: 1-2-3-4-5-10-1-11
 - Path 3: 1-2-3-6-8-9-10-1-11
 - Path 4: 1-2-3-6-7-9-10-1-11

- Note that each new path introduces a new edge.
- The path 1-2-3-4-5-10-1-2-3-6-8-9-10-1-11 is not considered to be an independent path because it is simply a combination of already specified paths and does not traverse any new edges.
- Paths 1 through 4 constitute a basis set for the flow graph in the previous figure.
- That is, if you can design tests to force execution of these paths (a basis set), every statement in the program will have been guaranteed to be executed at least one time and every condition will have been executed on its true and false sides.
- It should be noted that the basis set is not unique. In fact, a number of different basis sets can be derived for a given procedural design.

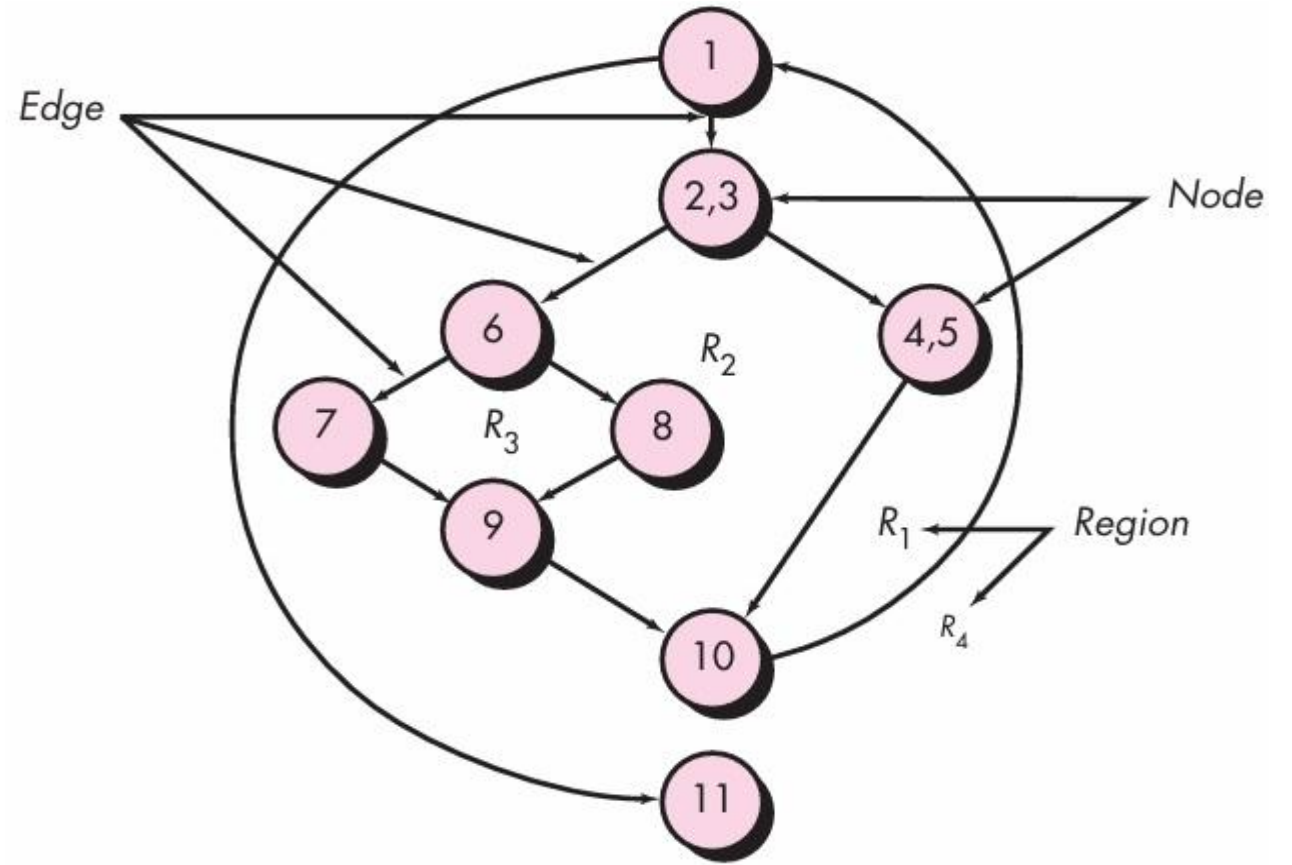
- How do you know how many paths to look for? The computation of cyclomatic complexity provides the answer.
- Cyclomatic complexity is a software metric that provides a quantitative measure of the logical complexity of a program.
- When used in the context of the basis path testing method, the value computed for cyclomatic complexity defines the number of independent paths in the basis set of a program and provides you with an upper bound for the number of tests that must be conducted to ensure that all statements have been executed at least once.

- Cyclomatic complexity has a foundation in graph theory and provides you with an extremely useful software metric. Complexity is computed in one of three ways:
- 1. The number of regions of the flow graph corresponds to the cyclomatic complexity.
- 2. Cyclomatic complexity $V(G)$ for a flow graph G is defined as
 - $V(G) = E - N + 2$ where E is the number of flow graph edges and N is the number of flow graph nodes.
- 3. Cyclomatic complexity $V(G)$ for a flow graph G is also defined as $V(G) = P + 1$ where P is the number of predicate nodes contained in the flow graph G .

Flow Graph Notation



(a)



(b)

- Referring once more to the flow graph in the previous figure, the cyclomatic complexity can be computed using each of the algorithms just noted:
- 1. The flow graph has four regions.
- 2. $V(G) = 11 \text{ edges} - 9 \text{ nodes} + 2 = 4$.
- 3. $V(G) = 3 \text{ predicate nodes} + 1 = 4$.
- Therefore, the cyclomatic complexity of the flow graph for the previous figure is 4.
- More important, the value for $V(G)$ provides you with an upper bound for the number of independent paths that form the basis set and, by implication, an upper bound on the number of tests that must be designed and executed to guarantee coverage of all program statements.

Deriving Test Cases

- Here we present basis path testing as a series of steps. The procedure *average*, depicted in the next figure, will be used as an example to illustrate each step in the test-case design method.
- Note that *average*, although an extremely simple algorithm, contains compound conditions and loops.
- The following steps can be applied to derive the basis set:
 - Using the design or code as a foundation, draw a corresponding flow graph
 - Determine the cyclomatic complexity of the resultant flow graph
 - Determine a basis set of linearly independent paths

Using the design or code as a foundation, draw a corresponding flow graph

PROCEDURE average;

- * This procedure computes the average of 100 or fewer numbers that lie between bounding values; it also computes the sum and the total number valid.

INTERFACE RETURNS average, total.input, total.valid;

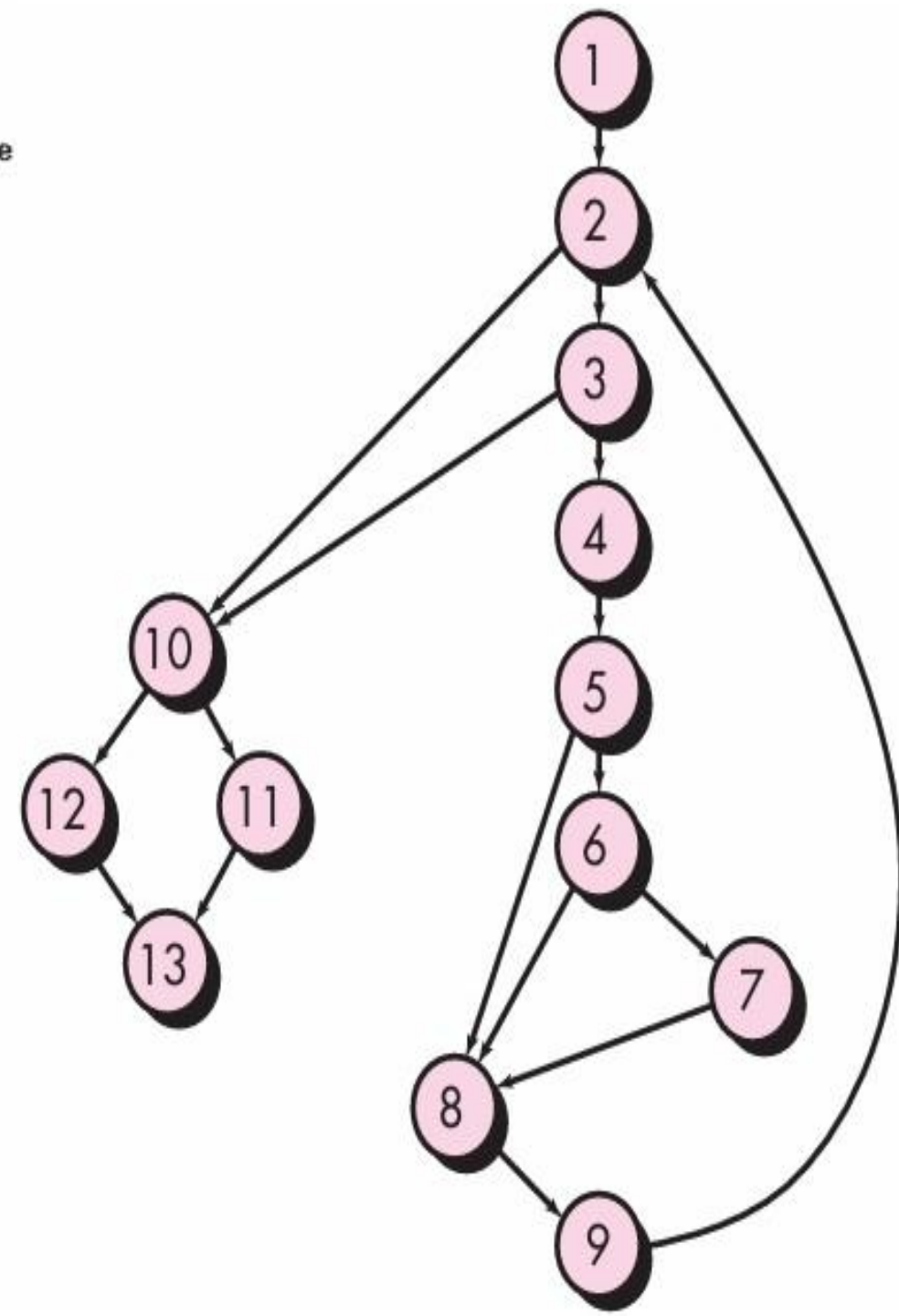
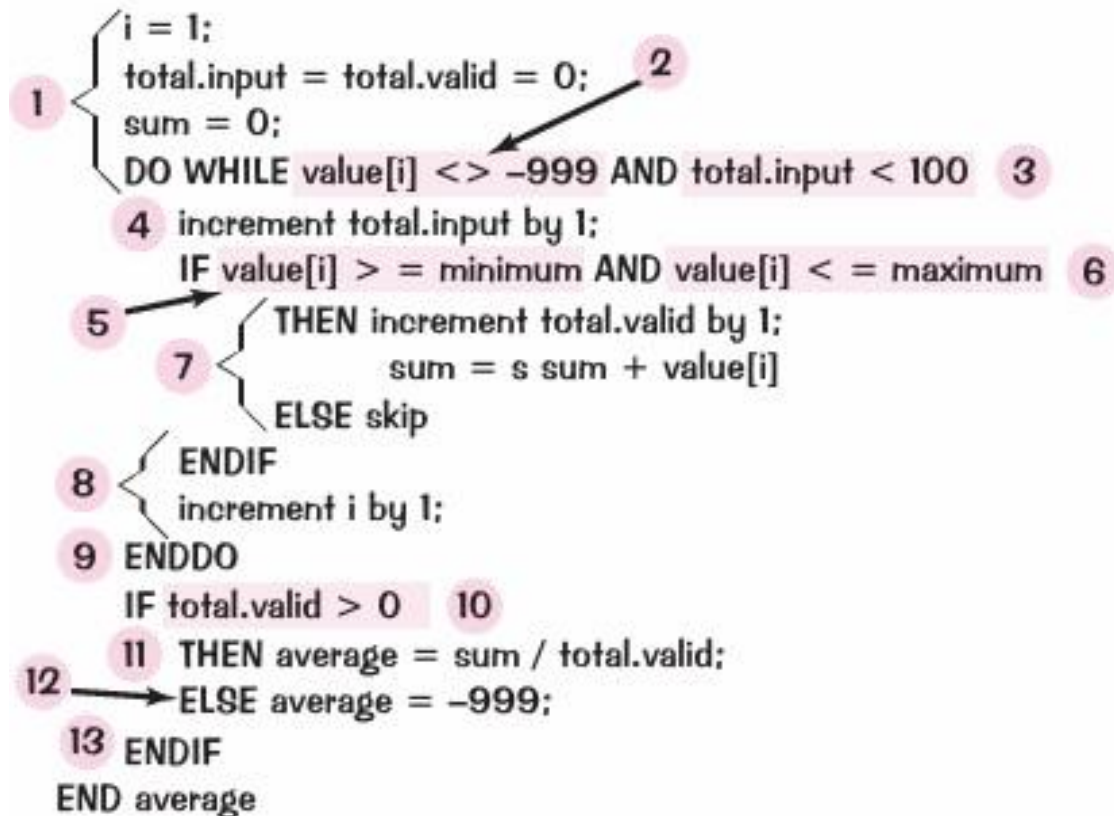
INTERFACE ACCEPTS value, minimum, maximum;

TYPE value[1:100] IS SCALAR ARRAY;

TYPE average, total.input, total.valid;

minimum, maximum, sum IS SCALAR;

TYPE i IS INTEGER;



Determine the cyclomatic complexity of the resultant flow graph

- The cyclomatic complexity $V(G)$ is determined by applying the algorithms described in the previous figure.
- $V(G) = 6$ regions
- $V(G) = 17 \text{ edges} - 13 \text{ nodes} + 2 = 6$
- $V(G) = 5 \text{ predicate nodes} + 1 = 6$

Determine a basis set of linearly independent paths

- The value of $V(G)$ provides the upper bound on the number of linearly independent paths through the program control structure. In the case of procedure *average*, we expect to specify six paths:
- Path 1: 1-2-10-11-13
- Path 2: 1-2-10-12-13
- Path 3: 1-2-3-10-11-13
- Path 4: 1-2-3-4-5-8-9-2-...
- Path 5: 1-2-3-4-5-6-8-9-2-...
- Path 6: 1-2-3-4-5-6-7-8-9-2-...

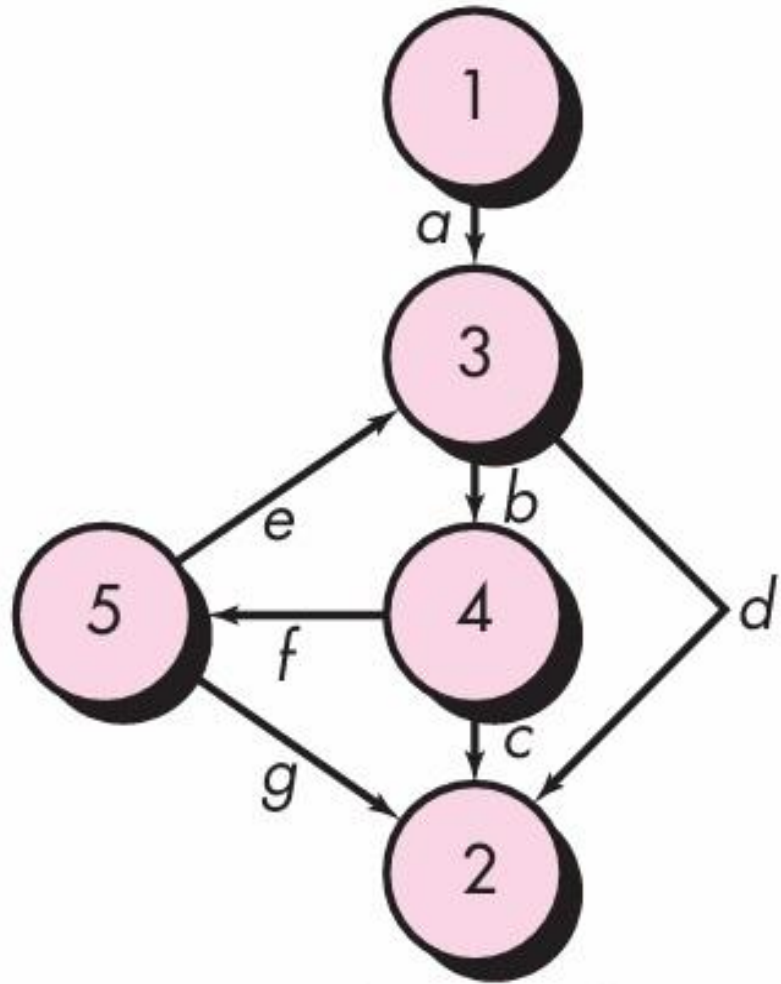
- The ellipsis (. . .) following paths 4, 5, and 6 indicates that any path through the remainder of the control structure is acceptable.
- It is often worthwhile to identify predicate nodes as an aid in the derivation of test cases. In this case, nodes 2, 3, 5, 6, and 10 are predicate nodes.

Prepare test cases that will force execution of each path in the basis set

- Data should be chosen so that conditions at the predicate nodes is tested. Each test case is executed and compared to expected results.
- Once all test cases have been completed, the tester can be sure that all statements in the program have been executed at least once.

Graph Matrices

- The procedure for deriving the flow graph and even determining a set of basis paths is amenable to mechanization. A data structure, called a graph matrix, can be quite useful for developing a software tool that assists in basis path testing.
- A graph matrix is a square matrix whose size (i.e., number of rows and columns) is equal to the number of nodes on the flow graph. Each row and column corresponds to an identified node, and matrix entries correspond to connections (an edge) between nodes. A simple example of a flow graph and its corresponding graph matrix is shown in the next figure.



Flow graph

Connected to node		1	2	3	4	5
Node						
1				<i>a</i>		
2						
3			<i>d</i>		<i>b</i>	
4			<i>c</i>			<i>f</i>
5			<i>g</i>	<i>e</i>		

Graph matrix

- To this point, the graph matrix is nothing more than a tabular representation of a flow graph. However, by adding a link weight to each matrix entry, the graph matrix can become a powerful tool for evaluating program control structure during testing.
- The link weight provides additional information about control flow. In its simplest form, the link weight is 1 (a connection exists) or 0 (a connection does not exist).
- But link weights can be assigned other, more interesting properties:
 - The probability that a link (edge) will be execute
 - The processing time expended during traversal of a link
 - The memory required during traversal of a link
 - The resources required during traversal of a link

Control Structure Testing

- The basis path testing technique described previously is one of a number of techniques for control structure testing.
- Although basis path testing is simple and highly effective, it is not sufficient in itself.
- Here, other variations on control structure testing are discussed. These broaden testing coverage and improve the quality of white-box testing.

Condition Testing

- Condition testing is a test-case design method that exercises the logical conditions contained in a program module.
- A simple condition is a Boolean variable or a relational expression, possibly preceded with one NOT ($\neg$) operator.
- A relational expression takes the form
 - $E1 <\text{relational-operator}> E2$
- If a condition is incorrect, then at least one component of the condition is incorrect.
- Therefore, types of errors in a condition include Boolean operator errors (incorrect/missing/extra Boolean operators), Boolean variable errors, Boolean parenthesis errors, relational operator errors, and arithmetic expression errors.
- The condition testing method focuses on testing each condition in the program to ensure that it does not contain errors.

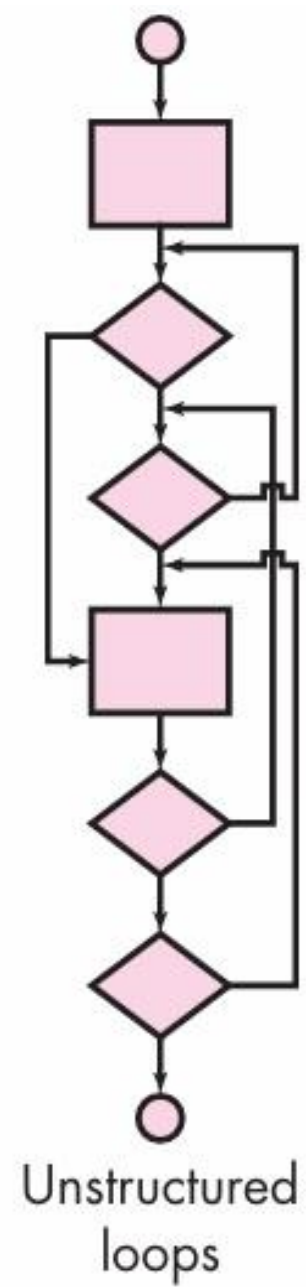
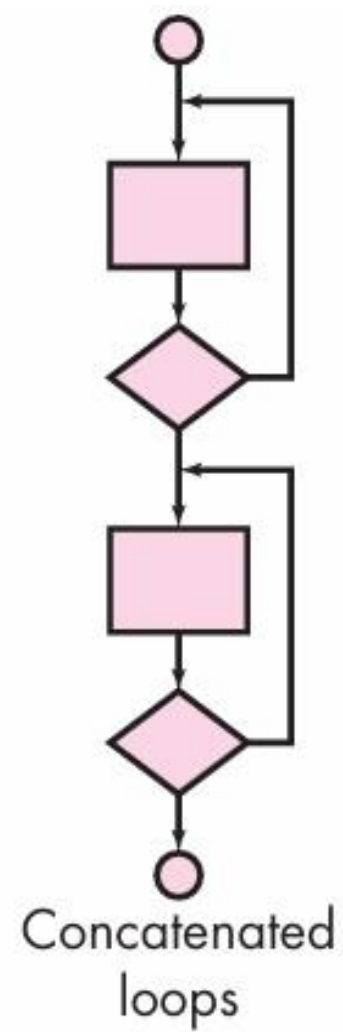
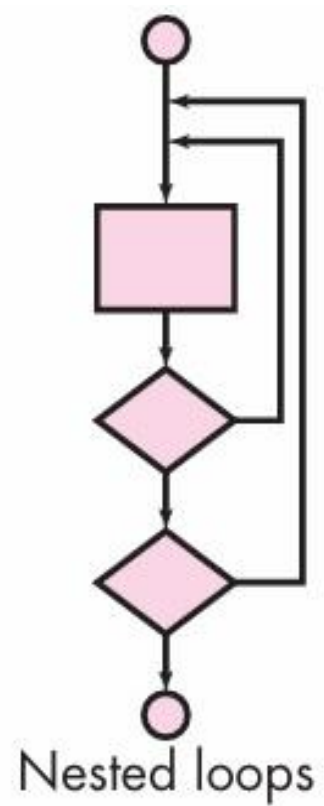
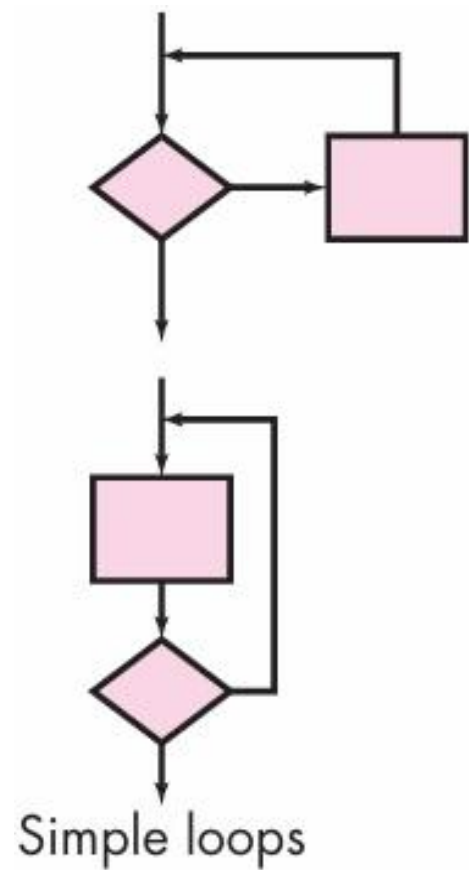
Data Flow Testing

- The data flow testing method selects test paths of a program according to the locations of definitions and uses of variables in the program.
- To illustrate the data flow testing approach, assume that each statement in a program is assigned a unique statement number. For a statement with S as its statement number,
 - $DEF(S) \{X \mid \text{statement } S \text{ contains a definition of } X\}$
 - $USE(S) \{X \mid \text{statement } S \text{ contains a use of } X\}$

- If statement S is an *if* or *loop* statement, its DEF set is empty and its USE set is based on the condition of statement S . The definition of variable X at statement S is said to be live at statement S' if there exists a path from statement S to statement S' that contains no other definition of X .
- A definition-use (DU) chain of variable X is of the form $[X, S, S']$, where S and S' are statement numbers, X is in $\text{DEF}(S)$ and $\text{USE}(S')$, and the definition of X in statement S is live at statement S' .
- One simple data flow testing strategy is to require that every DU chain be covered at least once. We refer to this strategy as the DU testing strategy.

Loop Testing

- Loops are the cornerstone for the vast majority of all algorithms implemented in software. And yet, we often pay them little heed while conducting software tests.
- Loop testing is a white-box testing technique that focuses exclusively on the validity of loop constructs.
- Four different classes of loops can be defined:
 - simple loops
 - concatenated loops
 - nested loops
 - unstructured loops



- Simple loops: the following set of tests can be applied to simple loops, where n is the maximum number of allowable passes through the loop.
 - 1. Skip the loop entirely
 - 2. Only one pass through the loop
 - 3. Two passes through the loop
 - 4. m passes through the loop where $m < n$
 - 5. $n - 1$, n , $n + 1$ passes through the loop

- Nested loops: if we were to extend the test approach for simple loops to nested loops, the number of possible tests would grow geometrically as the level of nesting increases.
- This would result in an impractical number of tests. Following approaches are suggested to reduce the number of tests.
 - 1. Start at the innermost loop. Set all other loops to minimum values
 - 2. Conduct simple loop tests for the innermost loop while holding the outer loops at their minimum iteration parameter (e.g., loop counter) values. Add other tests for out-of-range or excluded values.
 - 3. Work outward, conducting tests for the next loop, but keeping all other outer loops at minimum values and other nested loops to “typical” values
 - 4. Continue until all loops have been tested

- Concatenated loops: concatenated loops can be tested using the approach defined for simple loops, if each of the loops is independent of the other. However, if two loops are concatenated and the loop counter for loop 1 is used as the initial value for loop 2, then the loops are not independent. When the loops are not independent, the approach applied to nested loops is recommended.
- Unstructured loops: whenever possible, this class of loops should be redesigned to reflect the use of the structured programming constructs.

Black-Box Testing

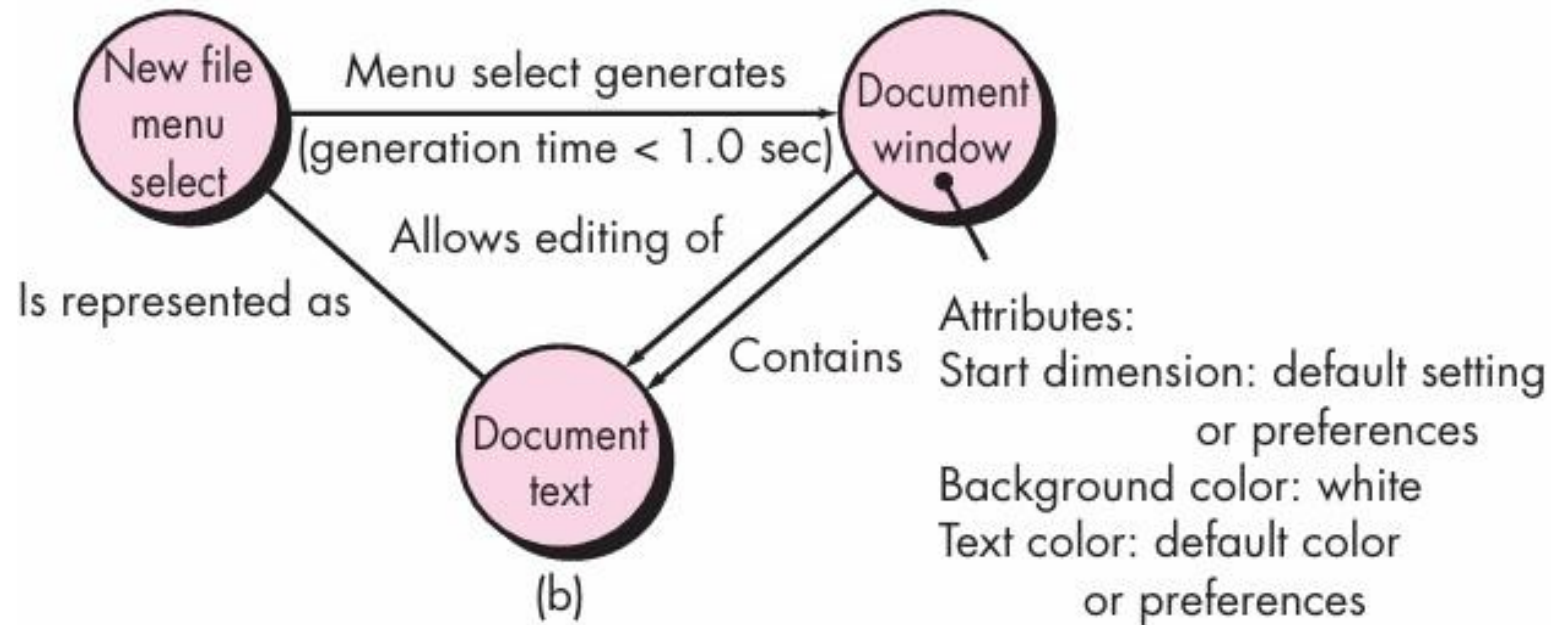
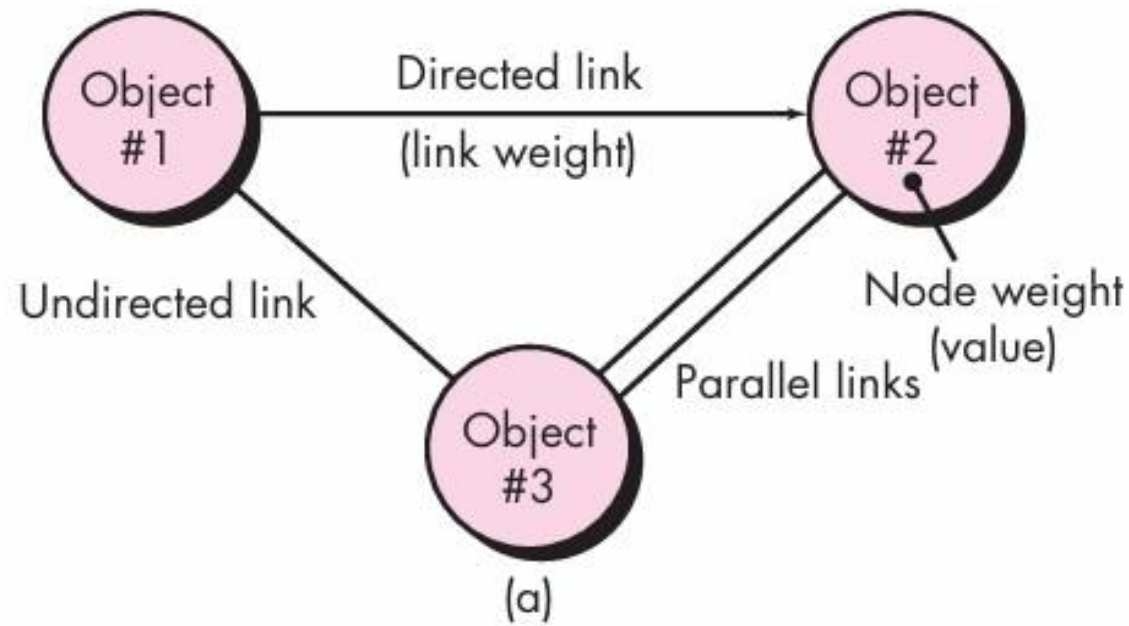
- Black-box testing, also called behavioral testing, focuses on the functional requirements of the software.
- That is, black-box testing techniques enable you to derive sets of input conditions that will fully exercise all functional requirements for a program.
- Black-box testing is not an alternative to white-box techniques. Rather, it is a complementary approach that is likely to uncover a different class of errors than white box methods.

- Black-box testing attempts to find errors in the following categories:
- (1) incorrect or missing functions
- (2) interface errors
- (3) errors in data structures or external database access
- (4) behavior or performance errors
- (5) initialization and termination errors.
- Because black-box testing purposely disregards control structure, attention is focused on the information domain.

Graph-Based Testing Methods

- The first step in black-box testing is to understand the objects that are modeled in software and the relationships that connect these objects. Once this has been accomplished, the next step is to define a series of tests that verify “all objects have the expected relationship to one another”.
- Stated in another way, software testing begins by creating a graph of important objects and their relationships and then devising a series of tests that will cover the graph so that each object and relationship is exercised and errors are uncovered.

- To accomplish these steps, you begin by creating a graph—a collection of nodes that represent objects, links that represent the relationships between objects, node weights that describe the properties of a node (e.g., a specific data value or state), and link weights that describe some characteristic of a link.
- The symbolic representation of a graph is shown in the next figure. Nodes are represented as circles connected by links that take a number of different forms. A directed link (represented by an arrow) indicates that a relationship moves in only one direction. A bidirectional link, also called a symmetric link, implies that the relationship applies in both directions. Parallel links are used when a number of different relationships are established between graph nodes.



Equivalence Partitioning

- Test-case design for equivalence partitioning is based on an evaluation of equivalence classes for an input condition. Using concepts introduced in the preceding section, if a set of objects can be linked by relationships that are symmetric, transitive, and reflexive, an equivalence class is present.
- An equivalence class represents a set of valid or invalid states for input conditions. Typically, an input condition is either a specific numeric value, a range of values, a set of related values, or a Boolean condition.

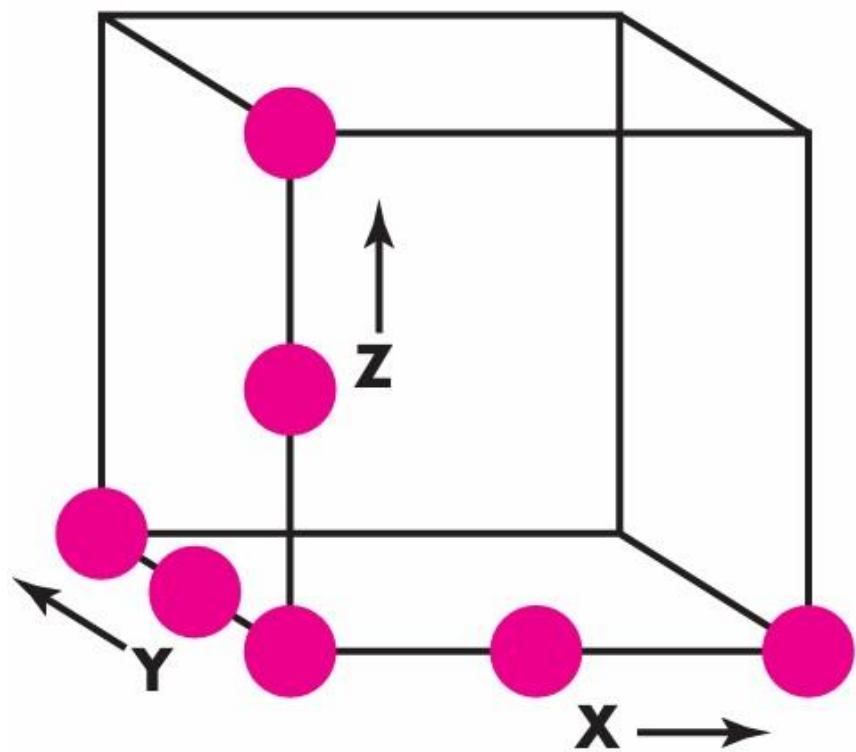
Boundary Value Analysis

- A greater number of errors occurs at the boundaries of the input domain rather than in the “center.”
- It is for this reason that boundary value analysis (BVA) has been developed as a testing technique.
- Boundary value analysis leads to a selection of test cases that exercise bounding values.

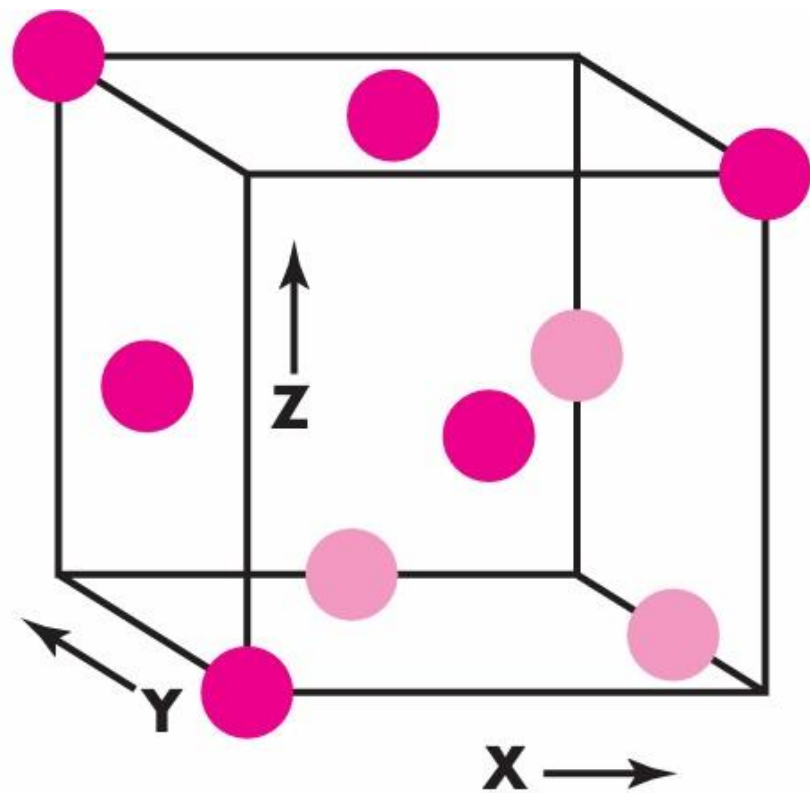
Orthogonal Array Testing

- There are many applications in which the input domain is relatively limited. That is, the number of input parameters is small and the values that each of the parameters may take are clearly bounded.
- When these numbers are very small (e.g., three input parameters taking on three discrete values each), it is possible to consider every input permutation and exhaustively test the input domain.
- However, as the number of input values grows and the number of discrete values for each data item increases, exhaustive testing becomes impractical or impossible.

- Orthogonal array testing can be applied to problems in which the input domain is relatively small but too large to accommodate exhaustive testing. The orthogonal array testing method is particularly useful in finding region faults—an error category associated with faulty logic within a software component.
- To illustrate the difference between orthogonal array testing and more conventional “one input item at a time” approaches, consider a system that has three input items, X, Y, and Z. Each of these input items has three discrete values associated with it. There are $3^3=27$ possible test cases.
- The next figure suggests a geometric view of the possible test cases associated with X, Y, and Z.
- Referring to the figure, one input item at a time may be varied in sequence along each input axis. This results in relatively limited coverage of the input domain (represented by the left-hand cube in the figure).



One input item at a time



L9 orthogonal array

- When orthogonal array testing occurs, an L9 orthogonal array of test cases is created. The L9 orthogonal array has a “balancing property”.
- That is, test cases (represented by dark dots in the figure) are “dispersed uniformly throughout the test domain,” as illustrated in the right-hand cube in the previous figure.
- Test coverage across the input domain is more complete.

To illustrate the use of the L9 orthogonal array, consider the *send* function for a fax application. Four parameters, P1, P2, P3, and P4, are passed to the *send* function. Each takes on three discrete values. For example, P1 takes on values:

P1 = 1, send it now

P1 = 2, send it one hour later

P1 = 3, send it after midnight

P2, P3, and P4 would also take on values of 1, 2, and 3, signifying other send functions.

If a “one input item at a time” testing strategy were chosen, the following sequence of tests (P1, P2, P3, P4) would be specified: (1, 1, 1, 1), (2, 1, 1, 1), (3, 1, 1, 1), (1, 2, 1, 1), (1, 3, 1, 1), (1, 1, 2, 1), (1, 1, 3, 1), (1, 1, 1, 2), and (1, 1, 1, 3). Phadke [Pha97] assesses these test cases by stating:

- Such test cases are useful only when one is certain that these test parameters do not interact. They can detect logic faults where a single parameter value makes the software malfunction. These faults are called single mode faults.
- This method cannot detect logic faults that cause malfunction when two or more parameters simultaneously take certain values; that is, it cannot detect any interactions. Thus its ability to detect faults is limited.
- Given the relatively small number of input parameters and discrete values, exhaustive testing is possible. The number of tests required is $3^4 = 81$, large but manageable. All faults associated with data item permutation would be found, but the effort required is relatively high.

- The orthogonal array testing approach enables you to provide good test coverage with far fewer test cases than the exhaustive strategy. An L9 orthogonal array for the fax send function is illustrated in the next figure.
- The result of tests using the L9 orthogonal array can be assessed in the following manner.
 - Detect and isolate all single mode faults
 - Detect all double mode faults
 - Multimode faults

Test case	Test parameters			
	P1	P2	P3	P4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1

